基于 STM32 的智能喂食器

设计与实现

Smart Pet Feeder

STM32 采集 + OneNET 云平台 + 微信小程序（含 AI 助手）

文档类型：系统设计文档

项目性质：物联网传输层 课程设计（2 人）

硬件平台：正点原子战舰 STM32F103ZET6

云端：中国移动 OneNET（物模型 + MQTT）

移动端：微信小程序（4 个页面 + AI 助手）

文档版本：V1.0

编写日期：2026 年 10 月

1 项目概述

1.1 背景

城市养宠人群规模持续增长，而“出差、加班、临时外出”是养宠家庭的常态。传统喂食器只能做定时投放，主人看不到粮食和水还剩多少；出门几天回来，常见的情况是粮仓空了、水碗干了，宠物已经饿了一两天。

这个项目要解决的是“人不在家时，还能看见并干预”的问题：设备侧持续采集粮仓余量、饮水余量和环境温湿度，云端保存数据，手机端随时查看；一旦余量低于阈值，手机端立即提示；主人也可以在手机上直接触发一次投食或加水。在此基础上又接了一个 AI 助手，用来回答“猫一天该吃多少克”这类养护问题。

1.2 设计目标

- **能看见**：粮仓余量（克）、饮水余量（百分比）、环境温湿度四类数据，手机端 3 秒刷新一次。
- **会提醒**：余粮 < 30 %、水位 < 30 %、温度越界（15–30 ℃）、湿度越界（40–80 %）时，页面高亮并给出提示。
- **能干预**：手机端可下发“手动投食 / 手动加水”，并带量参数。
- **有本地兜底**：断网时设备侧仍能显示数据、水位过低时本地声光告警，不依赖云。
- **低成本**：主控用现成的战舰开发板，传感器均为几十元级模块，整体硬件成本控制在 200 元以内。

1.3 分工与个人工作

项目由 2 人完成：一位同学负责硬件（选型、连线、设备端固件），我负责软件应用端与系统联调。我承担的部分：

- **小程序整体开发**：4 个页面（控制、监控、AI 助手、设置）的界面与交互，基于 TDesign 组件库搭建。
- **云端数据接入**：封装 OneNET HTTP API（属性查询、状态历史、属性设置、服务调用），统一错误处理与降级逻辑。
- **告警逻辑**：四类阈值的判定、页面级联更新、阈值本地持久化与跨页同步。

- **AI 助手**：接入小程序 agent-ui 组件与大模型，支持多轮对话、联网检索。
- **联调与文档**：与硬件同学对齐数据字段与上报频率，整理接口文档与测试记录。

2 需求分析

2.1 功能需求

表 2-1 功能需求清单

编号	需求	实现方式
F1	粮仓余量监测	HX711 + 称重传感器，去皮后换算克数
F2	饮水余量监测	YW01 液位传感器，ADC 采样换算百分比
F3	环境温湿度	DHT11 单总线读取
F4	本地显示与告警	LCD 实时刷新；水位 ≤ 20 % 时 LED 闪烁 + 蜂鸣 5 s
F5	数据上云	ESP8266（AT 指令）→ OneNET 物模型属性
F6	手机端查看	小程序 3 s 轮询属性接口，渲染数值与进度条
F7	阈值告警	四类阈值判定，页面高亮 + 顶部提示
F8	远程投食 / 加水	属性设置与服务调用两条通道，失败自动降级
F9	AI 养护问答	小程序 agent-ui + 大模型，多轮对话

2.2 性能与约束

表 2-2 性能指标与约束

指标	取值	说明
采集周期	100 ms	主循环 t%10 触发一次读取
上报周期	10 s	t%100 触发一次上报，避免频繁占用 WiFi
UI 刷新	3 s	小程序定时器轮询，兼顾实时性与请求量
余粮量程	0–4000 g	小程序按 4000 g 换算百分比
液位量程	0–100 %	低于 20 % 触发本地告警
温度告警区间	15–30 ℃	超出即提示
湿度告警区间	40–80 %	超出即提示

2.3 方案取舍

- **为什么用 OneNET 物模型而不是自定义 Topic**：物模型把“属性”抽象成统一 JSON，小程序端直接调 HTTP API 就能读写，不用自己维护 Topic 与报文格式；设备端也只需按 `$sys/{pid}/{dev}/thing/property/post` 发布。

- **为什么小程序用 HTTP 轮询而不是 WebSocket**：3 秒一次、一次几百字节，HTTP 足够；WebSocket 需要维持长连接，小程序切后台容易被回收，反而要处理重连。
- **为什么控制下发做了两条通道**：属性设置（`set-device-property`）适合“写状态”，服务调用（`call-service`）适合“触发动作”。不同设备/固件对两者的支持不一致，所以两条都试，前者失败自动降级到后者。

3 系统总体设计

3.1 三层架构

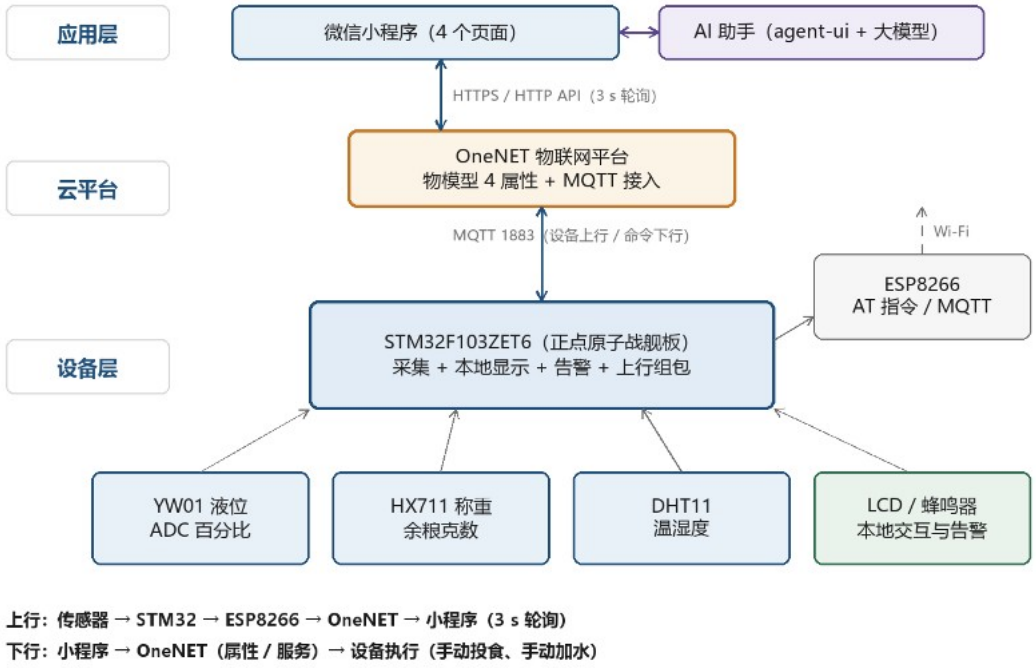


图 3-1 系统总体架构（设备层 / 云平台 / 应用层）

三层各自独立：设备层只负责采集、显示、组包；云平台负责存储与转发；应用层只面向用户。这样换传感器不影响小程序，换小程序也不影响设备。

3.2 数据流

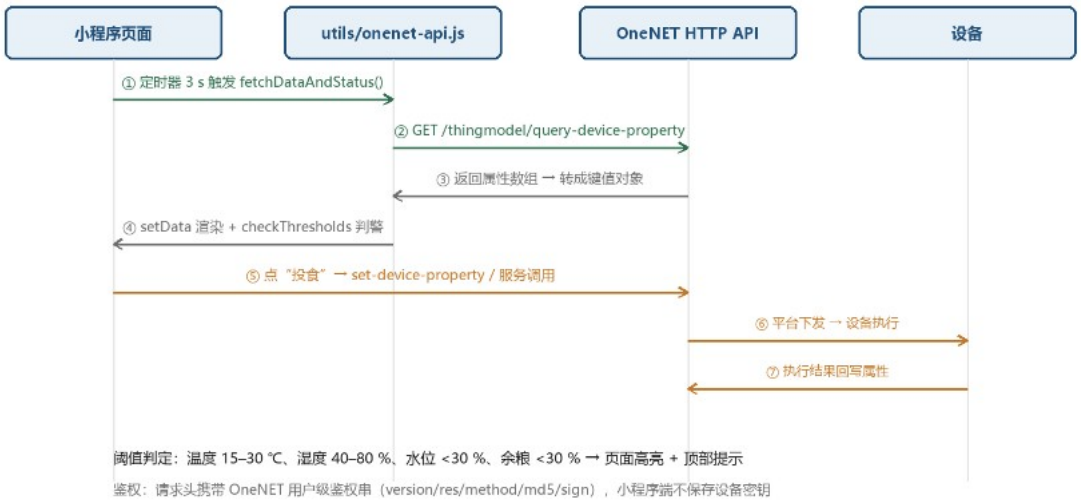


图 3-2 小程序端数据流（3 s 轮询 + 命令下发）

小程序侧的读取链路是“页面 → API 封装 → OneNET HTTP → 属性数组 → 转键值对象 → setData 渲染 → 判警”，每一步都有独立的错误处理：请求失败时页面保留上一次数据并标注“离线”，而不是清空。

4 硬件设计

4.1 器件清单

表 4-1 主要器件

类别	型号	作用	接口
主控	正点原子战舰 STM32F103ZET6	采集调度、LCD 显示、告警、组包	—
称重	HX711 + 应变式传感器	粮仓余量（克）	2 线（SCK/DOUT）
液位	YW01	饮水余量（0–100 %）	ADC
温湿度	DHT11	环境温度、湿度	单总线
通信	ESP8266（AT 固件）	Wi-Fi + MQTT 上云	USART2
显示	LCD（ILI93xx）	本地数据展示	FSMC
告警	蜂鸣器 + LED	水位过低声光提示	GPIO
交互	KEY0	重新校准皮重	GPIO

4.2 接口与引脚

表 4-2 接口与引脚分配

功能	外设	引脚 / 说明
调试串口	USART1	PA9 / PA10，115200，printf 重定向
ESP8266	USART2	PA2 / PA3，115200，AT 指令
HX711	GPIO	SCK / DOUT 两口，软件时序读取 24 位
YW01	ADC1	模拟输入，按满量程换算百分比
DHT11	单总线	一个 GPIO，主机拉低 ≥18 ms 启动
LCD	FSMC	16 位并口，ILI93xx 驱动
蜂鸣器 / LED	GPIO	推挽输出

4.3 HX711 称重与皮重校准

HX711 是 24 位 ADC，专为称重设计，内部集成稳压与时钟，外围只需两个 IO。读出的是原始码值，必须做两步处理：

- **去皮（Tare）**：空仓时连续采样取平均作为零点，存成 `offset`；之后每次读数减去 offset。
- **标定（Scale）**：放一个已知重量（如 500 g 砝码），用 `(读数 - offset) / 500` 得到比例系数，换算克数。

代码里上电时调用一次 `Get_Tare()`，并留了 KEY0 重校入口——因为粮仓拆装、传感器受力位置变化都会让零点漂移，现场没有按键重校功能的话，只能重新烧程序。

4.4 液位与本地告警

YW01 输出随水位变化的模拟量，ADC 采样后线性换算成 0–100 %。本地告警逻辑是“阈值 + 延时”的组合：

```
if (level <= 20) {
    LED1 = !LED1;              // 慢闪提示
    if (!beep_flag) {          // 只触发一次，避免长鸣
        BEEP = 1;
        beep_flag = 1;
        beep_timer = 500;      // 约 5 s 后自动关闭
    }
} else { LED1 = 1; BEEP = 0; beep_flag = 0; }
```

这里特意做的是“响 5 秒就停”，而不是一直响——喂食器放在卧室或阳台，长鸣会被主人直接拔电源，反而失去提醒意义。

5 设备端软件设计

5.1 主循环时序

设备端没有操作系统，主循环用“计数分频”的方式把不同周期的事拆开，避免用阻塞延时：

表 5-1 主循环分频时序

周期	动作	实现
10 ms	主循环节拍	`delay_ms(10)` + `t++`
100 ms	读 DHT11、YW01、HX711，刷新 LCD	`t % 10 == 0`
10 s	组 JSON、发 MQTT	`t % 100 == 0`，用 flag 保证只发一次
1 s	运行指示灯翻转	`t == 20` 时 `LED0` 取反

用分频而不是多个定时器，是因为这里对时序精度要求不高（100 ms 级），但代码更直观、更容易在现场改周期。

5.2 上行报文与主题

```
主题：  $sys/{产品 ID}/{设备名}/thing/property/post
载荷：  {"id":"123","params":{
          "food":{"value":1250},   // 余粮，克
          "shuiwei":{"value":65},  // 水位，%
          "temp":{"value":26},     // 温度，℃
          "humi":{"value":58}      // 湿度，%RH
        }}
```

报文按 OneNET 物模型的属性上报格式组织：`params` 下每个属性是 `{value: x}`。`id` 用固定字符串，平台只要求同一设备内唯一，用于匹配响应。

5.3 ESP8266 接入流程

ESP8266 用 AT 固件，设备端按固定顺序下发指令，每步之间留足延时等模块回显：

表 5-2 ESP8266 AT 接入流程

步骤	AT 指令	作用
1	`AT` / `AT+RST`	握手与复位
2	`AT+CWMODE=1`	Station 模式
3	`AT+CWDHCP=1,1`	开启 DHCP
4	`AT+CWJAP="SSID","PWD"`	连接 Wi-Fi（约 5 s）
5	`AT+MQTTUSERCFG=...`	配置 MQTT 客户端与鉴权
6	`AT+MQTTCONN=...`	连接 OneNET（1883）

步骤	AT 指令	作用
7	`AT+MQTTSUB=...`	订阅属性上报应答主题
8	`AT+MQTTPUB=...`	发布属性 JSON

5.4 实现状态说明

这一节如实说明各模块当前到什么程度，便于后续接手：

表 5-3 模块实现状态

模块	状态	说明
传感器采集	已验证	DHT11 / YW01 / HX711 读数正常，LCD 实时刷新
本地告警	已验证	水位 ≤ 20 % 触发 LED 闪烁 + 蜂鸣 5 s
LCD 显示	已验证	温度、湿度、液位、余粮四行实时刷新
MQTT 上行	编码完成，联调中	AT 序列与 JSON 组包已实现；受实验室 Wi-Fi 与 OneNET 参数影响，端到端透传尚未完成验证
小程序读取	已验证	3 s 轮询，四类阈值告警正常
控制下发	平台侧已验证	属性 / 服务两条通道均可下发，设备端执行待接执行机构
AI 助手	已验证	多轮对话、联网检索可用

把“联调中”标出来不是自曝其短，而是让文档可用：接手的人一眼知道从哪继续，而不是以为全部跑通了又找不到问题。

6 微信小程序设计

6.1 页面结构

表 6-1 小程序页面结构

页面	路径	职责
控制	pages/pet-feeder/control	实时数据卡片、手动投食/加水、阈值告警高亮
监控	pages/pet-feeder/monitor	余粮/水位进度、设备在线状态、加水历史记录
AI 助手	pages/pet-feeder/ai-assistant	养护问答（多轮对话 + 联网检索）
设置	pages/pet-feeder/settings	四类阈值配置，保存到本地缓存

四个页面通过 tabBar 平铺，不用多级路由——养宠场景下用户通常“看一眼就走”，层级越浅越好。

6.2 OneNET API 封装

所有云端请求收敛到 `utils/onenet-api.js` 一个文件，页面只调函数、不碰 URL 与鉴权：

```
fetchDeviceProperties()   GET  /thingmodel/query-device-property
fetchDeviceStatusHistory() GET  /device/status-history
setDeviceProperty(params)  POST /thingmodel/set-device-property
callDeviceService(name, p) POST /thingmodel/call-service
```

封装的收益有两个：一是换产品/设备只改一个 config 对象；二是错误统一在 Promise 里 reject，页面用 `.catch()` 处理，不会出现“某个页面忘了判 code 就渲染空数据”的问题。

6.3 轮询与判警

```
onLoad() {
  this.loadThresholds();          // 从本地缓存读阈值
  this.fetchDataAndStatus();      // 立即拉一次
  this.dataPolling = setInterval(() => this.fetchDataAndStatus(), 3000);
}
onUnload() { clearInterval(this.dataPolling); }   // 必须清，否则切页后仍在请求
```

判警逻辑放在 `checkThresholds(properties)` 里，四类阈值各自产出布尔值，再合并成页面状态。阈值从 `wx.getStorageSync("settings")` 读取，设置页保存后，控制页在 `onShow` 里重新加载——这样“设置里改了阈值、控制页还是旧的”这种不一致就不会出现。

6.4 控制下发与降级

“投食”按钮同时准备两条通道，第一条失败自动走第二条：

```
const p1 = setDeviceProperty({ manual_feed: true, feed_amount: 150 });
const p2 = callDeviceService("manual_feed", { amount: 150 });
p1.then(ok).catch(() => p2.then(ok).catch(fail));
```

这不是“多此一举”：不同固件对“属性”和“服务”的支持不一样，只走一条时，遇到不支持的固件就表现为“点了没反应”，而用户无法判断是网络问题还是设备问题。两条都试，至少能把“指令是否到达平台”这件事确定下来。

6.5 AI 助手

AI 页面用小程序官方的 agent-ui 组件，配置 `botId` 与模型供应商即可支持多轮对话、上传图片、联网检索。它不是“设备控制”的一部分，定位是**售后与养护咨询**：主人问“三个月大的布偶一天喂多少克”，不需要再去搜帖子。

7 云平台数据模型

7.1 物模型属性

表 7-1 物模型属性定义

标识符	名称	类型	单位	来源
food	余粮	int	g	HX711
shuiwei	水位	int	%	YW01
temp	温度	int	℃	DHT11
humi	湿度	int	%RH	DHT11

7.2 鉴权与安全

小程序请求 OneNET 时，请求头带的是**用户级鉴权串**（`version` / `res` / `method` / `md5` / `sign`），不是设备密钥；设备侧 MQTT 用的是设备三元组。两者分离的好处是：小程序端不持有设备密钥，泄露风险更低。

需要说明的是：鉴权串带有效期（`et`），过期后需要重新生成；当前实现把它写在 `config` 常量里，长期运行需要改成服务端下发或云函数代签——这是已知的安全改进点（见 9.2）。

8 系统测试

8.1 设备端测试

表 8-1 设备端测试

测试项	方法	结果
传感器读数	LCD 对照串口打印，改变液位/重量/温湿度	读数随输入变化，量程内线性
称重标定	空仓去皮 → 放 500 g 砝码	误差在 ±10 g 内（受传感器与摆放位置影响）
本地告警	水位降到 20 % 以下	LED 闪烁，蜂鸣 5 s 后自动停止
LCD 刷新	连续运行 2 h	无花屏、无卡死

8.2 小程序测试

表 8-2 小程序测试

测试项	方法	结果
属性读取	打开控制页，观察 3 s 刷新	数值与云端一致，断网时保留上次数据
阈值告警	设置页改阈值 → 返回控制页	告警状态立即跟随新阈值
控制下发	点“投食”，观察平台侧属性变化	指令到达平台，返回成功提示
AI 助手	连续多轮提问	上下文保持，联网检索可用
页面切换	四个 tab 来回切换 20 次	定时器正常清理，无重复请求

8.3 端到端联调现状

硬件采集与本地显示已稳定；小程序侧的数据读取、判警、下发链路完整可演示。设备端 MQTT 上行目前停在“AT 序列 + JSON 组包完成、透传待验证”这一步，主要卡在实验室 Wi-Fi 认证方式（Portal 认证无法用 `AT+CWJAP` 直接连）与 OneNET 参数配置上。这也是第 9 章第一条改进项。

9 局限与改进

9.1 当前局限

- **上云链路未闭环**：ESP8266 透传未在真实网络下验证通过，设备端数据尚未自动进入云端。
- **无执行机构**：投食/加水目前只到“指令下发”，没有舵机或水泵执行，属于半闭环。
- **无断网缓存**：设备端断网期间数据直接丢弃，恢复后不补传。
- **鉴权硬编码**：小程序端鉴权串写死在常量里，过期需手动改，存在泄露与失效风险。
- **无喂食记录**：只存了本地加水记录，缺少“谁在什么时候投了多少”的完整流水。

9.2 改进路线

表 9-1 改进路线

优先级	改进项	做法
高	打通行链路	改用手机热点或企业级 Wi-Fi（非 Portal），补齐 AT 回显判断与重连
高	加执行机构	SG90 舵机（出粮）+ 5 V 小水泵（加水），由属性事件触发定时动作
中	断网缓存	设备端用 Flash 环形缓冲存最近 N 帧，重连后按时间序补发
中	鉴权改造	鉴权串由云函数按需签发，小程序只拿短期 token
低	喂食流水	云端建“投食记录”数据流，小程序做日/周统计

附录 A 代码目录

```
feeder/
├── hardware/                # 设备端（Keil MDK，正点原子战舰板）
│   ├── USER/test.c          # 主程序：采集、显示、告警、上报
│   ├── HARDWARE/HX711/      # 称重驱动
│   ├── HARDWARE/YW01/       # 液位驱动
│   ├── HARDWARE/DHT11/      # 温湿度驱动
│   ├── NET/ESP8266/         # AT 指令封装
│   └── NET/ONENET/          # 平台接入
└── miniprogram/             # 小程序（TDesign）
    ├── pages/pet-feeder/    # control / monitor / ai-assistant / settings
    └── utils/onenet-api.js  # 云端 API 封装
```

附录 B 配置占位说明

仓库中的 Wi-Fi 账号密码、OneNET 产品 ID / 设备名 / 鉴权串均已替换为占位符。本地运行时需要新建 `config.local.h`（已加入 `.gitignore`）填入真实值，或直接在代码中替换占位符。

```
#define WIFI_SSID       "YOUR_WIFI_SSID"
#define WIFI_PASSWORD   "YOUR_WIFI_PASSWORD"
#define MQTT_USERNAME   "YOUR_DEVICE_NAME"
// 主题：  $sys/{产品 ID}/{设备名}/thing/property/post
```

本文档依据项目源码（设备端 Keil 工程、小程序源码）与《基于 STM32 的智能喂食器设计与实现》整理；实现状态按代码实际完成度标注，未把设计目标写成已实现功能。